Experiment No:- 3

Name:- Dipesh Makdiya
Class:- D15C
Roll no:- 32

EXPERIMENT: Decision Tree & Random Forest for Classification

---

1. Dataset Source

Dataset Used: Iris Dataset

Type: Classification

Source:
https://scikit-learn.org/stable/datasets/toy_dataset.html#iris-dataset

Access Method:
```
from sklearn.datasets import load_iris
```

Reason for Selection:

- Real-world benchmark dataset
- Suitable for classification problems
- Commonly used for implementing tree-based algorithms

---

2. Dataset Description

2.1 Iris Dataset

Objective:
To classify iris flowers into different species using Decision Tree and Random Forest classifiers.

Number of Instances: 150
Number of Features: 4

Features:

- Sepal length
- Sepal width
- Petal length
- Petal width

Target Variable:

Species:

- Setosa
- Versicolor
- Virginica

Characteristics:

- Multiclass classification dataset
- No missing values
- Numerical feature values
- Linearly separable for some classes
- Suitable for tree-based models

---

3. Mathematical Formulation of the Algorithms

3.1 Decision Tree Classifier

A Decision Tree recursively splits the dataset based on the feature that provides the best class separation.

Gini Index:

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Entropy:

$$Entropy = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Information Gain:

$$IG = Entropy(parent) - \sum_{k} \frac{n_k}{n} Entropy(child_k)$$

Where:

- pipi = probability of class ii
- nknk = number of samples in child node
- nn = total number of samples

The feature with the **maximum Information Gain** is selected for splitting.

```
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

3.2 Random Forest Classifier

Random Forest is an ensemble learning technique that constructs multiple decision trees and combines their outputs.

Key Concepts:

- Bootstrap sampling (sampling with replacement)
- Random subset of features for each split
- Majority voting for final prediction

Final Prediction:

$$\hat{y} = mode(h_1(x), h_2(x), ..., h_n(x))$$

Where:
$hi(x)$hi(x) is the prediction of each individual decision tree.

---

4. Algorithm Limitations

Decision Tree Limitations

- Prone to overfitting
- Sensitive to small variations in training data
- Deep trees become complex and less interpretable

Random Forest Limitations

- Computationally more expensive
- Requires more memory
- Less interpretable compared to a single decision tree
- Needs hyperparameter tuning for best performance

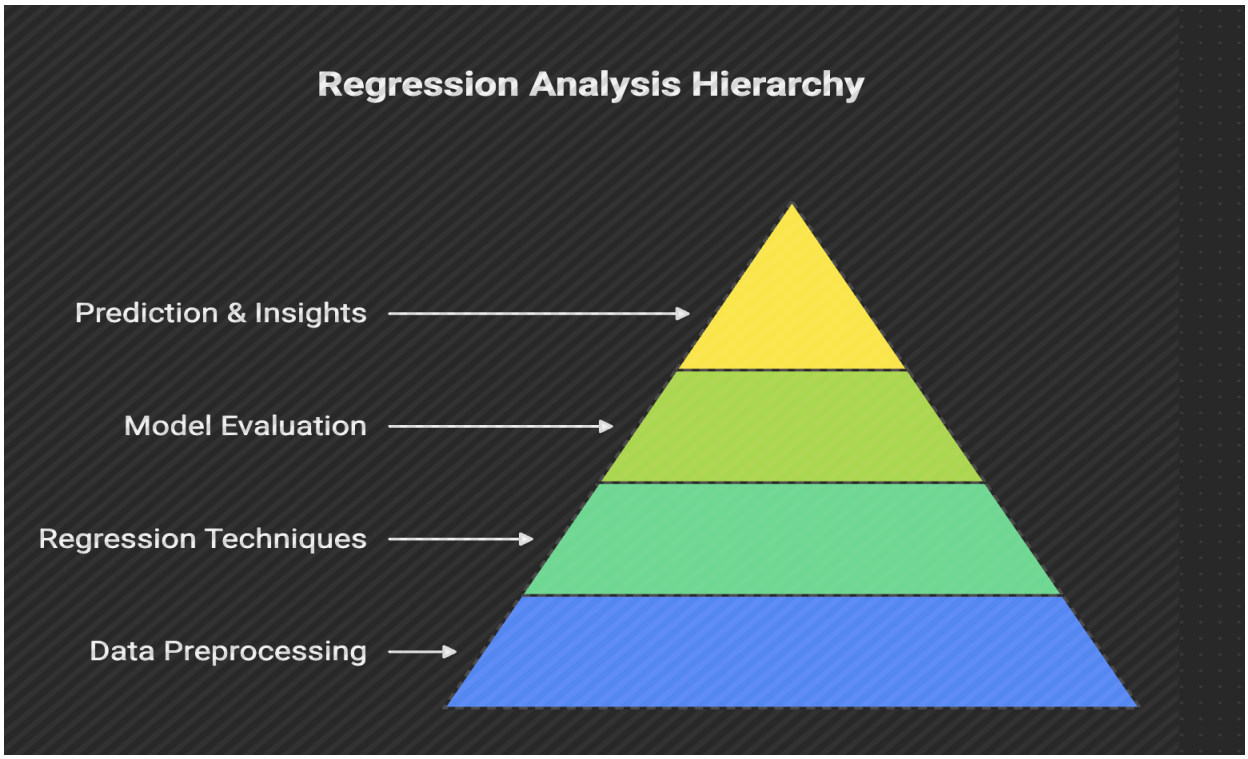
---

5. Methodology / Workflow

Step-by-Step Workflow:

1. Import required libraries
2. Load the Iris dataset
3. Data exploration
4. Feature and target selection
5. Train-test split
6. Train Decision Tree classifier
7. Train Random Forest classifier
8. Perform prediction
9. Evaluate model performance
10. Compare results

Workflow Diagram:



Dataset → Preprocessing → Train/Test Split →
Model Training → Prediction → Evaluation

---

6. Performance Analysis

Evaluation Metrics Used:

- Accuracy
- Confusion Matrix
- Classification Report
    - Precision
    - Recall
    - F1-score

Interpretation:

- Higher accuracy indicates better classification performance
- Confusion matrix shows correct and incorrect predictions
- Precision measures correctness of positive predictions
- Recall measures model's ability to find all positive samples
- F1-score balances precision and recall

Random Forest generally provides better performance because:

- It reduces overfitting
- It combines multiple trees for robust prediction

---

7. Hyperparameter Tuning

Decision Tree

Tuned Parameters:

- max_depth
- criterion (gini / entropy)
- min_samples_split

Impact:

- Controls tree complexity

- Prevents overfitting
- Improves generalization

Random Forest

Tuned Parameters:

- n_estimators
- max_depth
- max_features

Impact:

- Increases model accuracy
- Reduces variance
- Improves overall performance

---

8. Google Colab Link

Experiment 3.ipynb

---

9. Result

- Decision Tree classifier achieved good classification accuracy on the Iris dataset.
- Random Forest classifier achieved higher accuracy and better generalization due to ensemble learning.

---

10. Conclusion

Decision Tree and Random Forest algorithms were successfully implemented for multiclass classification using the Iris dataset.

Random Forest outperformed the Decision Tree in terms of accuracy, robustness, and resistance to overfitting.
This experiment demonstrates the effectiveness of ensemble learning methods in supervised machine learning.